An array stores multiple values in one single variable:

Example

```php
$cars = array("Volvo", "BMW", "Toyota");
```

Try it Yourself »

What is an Array?

An array is a special variable that can hold many values under a single name, and you can access the values by referring to an index number or name.

PHP Array Types

In PHP, there are three types of arrays:

- **Indexed arrays** - Arrays with a numeric index
- **Associative arrays** - Arrays with named keys
- **Multidimensional arrays** - Arrays containing one or more arrays

---

Working With Arrays

In this tutorial you will learn how to work with arrays, including:

- Create Arrays
- Access Arrays
- Update Arrays
- Add Array Items
- Remove Array Items
- Sort Arrays

Array Items

Array items can be of any data type.

The most common are strings and numbers (int, float), but array items can also be objects, functions or even arrays.

You can have different data types in the same array.

Example

Array items of four different data types:

```
$myArr = array("Volvo", 15, ["apples", "bananas"], myFunction);
```

Try it Yourself »

Array Functions

The real strength of PHP arrays are the built-in array functions, like the `count()` function for counting array items:

Example

How many items are in the `$cars` array:

```
$cars = array("Volvo", "BMW", "Toyota");

echo count($cars);
```

PHP Indexed Arrays

In indexed arrays each item has an index number.

By default, the first item has index 0, the second item has item 1, etc.

Example

Create and display an indexed array:

```php
$cars = array("Volvo", "BMW", "Toyota");

var_dump($cars);
```

Access Indexed Arrays

To access an array item you can refer to the index number.

Example

Display the first array item:

```php
$cars = array("Volvo", "BMW", "Toyota");

echo $cars[0];
```

Change Value

To change the value of an array item, use the index number:

Example

Change the value of the second item:

```php
$cars = array("Volvo", "BMW", "Toyota");

$cars[1] = "Ford";

var_dump($cars);
```

Loop Through an Indexed Array

To loop through and print all the values of an indexed array, you could use a `foreach` loop, like this:

Example

Display all array items:

```php
$cars = array("Volvo", "BMW", "Toyota");
```

```php
foreach ($cars as $x) {
  echo "$x <br>";
}
```

Index Number

The key of an indexed array is a number, by default the first item is 0 and the second is 1 etc., but there are exceptions.

New items get the next index number, meaning one higher than the *highest existing index*.

So if you have an array like this:

```php
$cars[0] = "Volvo";
$cars[1] = "BMW";
$cars[2] = "Toyota";
```

And if you use the `array_push()` function to add a new item, the new item will get the index 3:

Example

```php
array_push($cars, "Ford");
var_dump($cars);
```

But if you have an array with random index numbers, like this:

```php
$cars[5] = "Volvo";
$cars[7] = "BMW";
$cars[14] = "Toyota";
```

And if you use the `array_push()` function to add a new item, what will be the index number of the new item?

Example

```php
array_push($cars, "Ford");
```

```php
var_dump($cars);
```

PHP Associative Arrays

Associative arrays are arrays that use named keys that you assign to them.

Example

```php
$car = array("brand"=>"Ford", "model"=>"Mustang", "year"=>1964);

var_dump($car);
```

Access Associative Arrays

To access an array item you can refer to the key name.

Example

Display the model of the car:

```php
$car = array("brand"=>"Ford", "model"=>"Mustang", "year"=>1964);

echo $car["model"];
```

Change Value

To change the value of an array item, use the key name:

Example

Change the year item:

```php
$car = array("brand"=>"Ford", "model"=>"Mustang", "year"=>1964);

$car["year"] = 2024;

var_dump($car);
```

Loop Through an Associative Array

To loop through and print all the values of an associative array, you could use a `foreach` loop, like this:

Example

Display all array items, keys and values:

```
$car = array("brand"=>"Ford", "model"=>"Mustang", "year"=>1964);


foreach ($car as $x => $y) {
  echo "$x: $y <br>";
}
```

Create Array

You can create arrays by using the `array()` function:

Example

```
$cars = array("Volvo", "BMW", "Toyota");
```

Try it Yourself »

You can also use a shorter syntax by using the `[]` brackets:

Example

```
$cars = ["Volvo", "BMW", "Toyota"];
```

Try it Yourself »

Multiple Lines

Line breaks are not important, so an array declaration can span multiple lines:

Example

```php
$cars = [
  "Volvo",
  "BMW",
  "Toyota"
];
```

Try it Yourself »

Trailing Comma

A comma after the last item is allowed:

Example

```php
$cars = [
  "Volvo",
  "BMW",
  "Toyota",
];
```

Try it Yourself »

Array Keys

When creating indexed arrays the keys are given automatically, starting at 0 and increased by 1 for each item, so the array above could also be created with keys:

Example

```php
$cars = [
```

```
  0 => "Volvo",
  1 => "BMW",
  2 =>"Toyota"
];
```

Try it Yourself »

As you can see, indexed arrays are the same as associative arrays, but associative arrays have names instead of numbers:

Example

```
$myCar = [
  "brand" => "Ford",
  "model" => "Mustang",
  "year" => 1964
];
```

Try it Yourself »

Declare Empty Array

You can declare an empty array first, and add items to it later:

Example

```
$cars = [];
$cars[0] = "Volvo";
$cars[1] = "BMW";
$cars[2] = "Toyota";
```

Try it Yourself »

The same goes for associative arrays, you can declare the array first, and then add items to it:

Example

```php
$myCar = [];

$myCar["brand"] = "Ford";

$myCar["model"] = "Mustang";

$myCar["year"] = 1964;
```

Try it Yourself »

Mixing Array Keys

You can have arrays with both indexed and named keys:

Example

```php
$myArr = [];

$myArr[0] = "apples";

$myArr[1] = "bananas";

$myArr["fruit"] = "cherries";
```

Try it Yourself »

Access Array Item

To access an array item, you can refer to the index number for indexed arrays, and the key name for associative arrays.

Example

Access an item by referring to its index number:

```php
$cars = array("Volvo", "BMW", "Toyota");
```

```php
echo $cars[2];
```

Try it Yourself »

Note: The first item has index 0.

To access items from an **associative array**, use the key name:

Example

Access an item by referring to its key name:

```php
$cars = array("brand" => "Ford", "model" => "Mustang", "year" => 1964);

echo $cars["year"];
```

Try it Yourself »

Double or Single Quotes

You can use both double and single quotes when accessing an array:

Example

```php
echo $cars["model"];

echo $cars['model'];
```

Try it Yourself »

Excecute a Function Item

Array items can be of any data type, including function.

To execute such a function, use the index number followed by parentheses `()`:

Example

Execute a function item:

```php
function myFunction() {
  echo "I come from a function!";
}

$myArr = array("Volvo", 15, myFunction);

$myArr[2]();
```

Try it Yourself »

Use the key name when the function is an item in a associative array:

Example

Execute function by referring to the key name:

```php
function myFunction() {
  echo "I come from a function!";
}

$myArr = array("car" => "Volvo", "age" => 15, "message" => myFunction);

$myArr["message"]();
```

Try it Yourself »

Loop Through an Associative Array

To loop through and print all the values of an associative array, you can use a `foreach` loop, like this:

Example

Display all array items, keys and values:

```php
$car = array("brand"=>"Ford", "model"=>"Mustang", "year"=>1964);


foreach ($car as $x => $y) {
  echo "$x: $y <br>";
}
```

Try it Yourself »

Loop Through an Indexed Array

To loop through and print all the values of an indexed array, you can use a `foreach` loop, like this:

Example

Display all array items:

```php
$cars = array("Volvo", "BMW", "Toyota");
foreach ($cars as $x) {
  echo "$x <br>";
}
```

Update Array Item

To update an existing array item, you can refer to the index number for indexed arrays, and the key name for associative arrays.

Example

Change the second array item from "BMW" to "Ford":

```php
$cars = array("Volvo", "BMW", "Toyota");

$cars[1] = "Ford";
```

Try it Yourself »

Note: The first item has index 0.

To update items from an **associative array**, use the key name:

Example

Update the year to 2024:

```php
$cars = array("brand" => "Ford", "model" => "Mustang", "year" => 1964);

$cars["year"] = 2024;
```

Try it Yourself »

Update Array Items in a Foreach Loop

There are different techniques to use when changing item values in a `foreach` loop.

One way is to insert the `&` character in the assignment to assign the item value by reference, and thereby making sure that any changes done with the array item inside the loop will be done to the original array:

Example

Change ALL item values to "Ford":

```php
$cars = array("Volvo", "BMW", "Toyota");

foreach ($cars as &$x) {

  $x = "Ford";

}

unset($x);
```

```php
var_dump($cars);
```

Try it Yourself »

Note: Remember to add the `unset()` function after the loop.

Without the `unset($x)` function, the `$x` variable will remain as a reference to the last array item.

To demonstrate this, see what happens when we change the value of `$x` after the `foreach` loop:

Example

Demonstrate the consequence of forgetting the `unset()` function:

```php
$cars = array("Volvo", "BMW", "Toyota");

foreach ($cars as &$x) {

  $x = "Ford";

}


$x = "ice cream";


var_dump($cars);
```

Try it Yourself »

Add Array Item

To add items to an existing array, you can use the bracket `[]` syntax.

Example

Add one more item to the `fruits` array:

```php
$fruits = array("Apple", "Banana", "Cherry");

$fruits[] = "Orange";
```

Try it Yourself »

Associative Arrays

To add items to an associative array, or key/value array, use brackets `[]` for the key, and assign value with the `=` operator.

Example

Add one item to the `car` array:

```
$cars = array("brand" => "Ford", "model" => "Mustang");

$cars["color"] = "Red";
```

Try it Yourself »

Add Multiple Array Items

To add multiple items to an existing array, use the `array_push()` function.

Example

Add three item to the `fruits` array:

```
$fruits = array("Apple", "Banana", "Cherry");

array_push($fruits, "Orange", "Kiwi", "Lemon");
```

Try it Yourself »

Add Multiple Items to Associative Arrays

To add multiple items to an existing array, you can use the `+=` operator.

Example

Add two items to the `cars` array:

```php
$cars = array("brand" => "Ford", "model" => "Mustang");

$cars += ["color" => "red", "year" => 1964];
```

Try it Yourself »

Remove Array Item

To remove an existing item from an array, you can use the `array_splice()` function.

With the `array_splice()` function you specify the index (where to start) and how many items you want to delete.

Example

Remove the second item:

```php
$cars = array("Volvo", "BMW", "Toyota");

array_splice($cars, 1, 1);
```

Try it Yourself »

After the deletion, the array gets reindexed automatically, starting at index 0.

Using the unset Function

You can also use the `unset()` function to delete existing array items.

Note: The `unset()` function does not re-arrange the indexes, meaning that after deletion the array will no longer contain the missing indexes.

Example

Remove the second item:

```php
$cars = array("Volvo", "BMW", "Toyota");

unset($cars[1]);
```

Try it Yourself »

Remove Multiple Array Items

To remove multiple items, the `array_splice()` function takes a length parameter that allows you to specify the number of items to delete.

Example

Remove 2 items, starting a the second item (index 1):

```php
$cars = array("Volvo", "BMW", "Toyota");

array_splice($cars, 1, 2);
```

Try it Yourself »

The `unset()` function takes a unlimited number of arguments, and can therefore be used to delete multiple array items:

Example

Remove the first and the second item:

```php
$cars = array("Volvo", "BMW", "Toyota");

unset($cars[0], $cars[1]);
```

Try it Yourself »

Remove Item From an Associative Array

To remove items from an associative array, you can use the `unset()` function.

Specify the key of the item you want to delete.

Example

Remove the "model":

```
$cars = array("brand" => "Ford", "model" => "Mustang", "year" => 1964);

unset($cars["model"]);
```

Try it Yourself »

Using the array_diff Function

You can also use the `array_diff()` function to remove items from an associative array.

This function returns a new array, without the specified items.

Example

Create a new array, without "Mustang" and "1964":

```
$cars = array("brand" => "Ford", "model" => "Mustang", "year" => 1964);

$newarray = array_diff($cars, ["Mustang", 1964]);
```

Try it Yourself »

Note: The `array_diff()` function takes *values* as parameters, and not *keys*.

Remove the Last Item

The `array_pop()` function removes the last item of an array.

Example

Remove the last item:

```
$cars = array("Volvo", "BMW", "Toyota");
```

```
array_pop($cars);
```

Try it Yourself »

Remove the First Item

The `array_shift()` function removes the first item of an array.

Example

Remove the first item:

```
$cars = array("Volvo", "BMW", "Toyota");

array_shift($cars);
```

Try it Yourself »

PHP Sorting Arrays

❮ PreviousNext ❯

The elements in an array can be sorted in alphabetical or numerical order, descending or ascending.

PHP - Sort Functions For Arrays

In this chapter, we will go through the following PHP array sort functions:

- `sort()` - sort arrays in ascending order
- `rsort()` - sort arrays in descending order
- `asort()` - sort associative arrays in ascending order, according to the value

- ksort() - sort associative arrays in ascending order, according to the key
- arsort() - sort associative arrays in descending order, according to the value
- krsort() - sort associative arrays in descending order, according to the key

Sort Array in Ascending Order - sort()

The following example sorts the elements of the $cars array in ascending alphabetical order:

Example

```
$cars = array("Volvo", "BMW", "Toyota");

sort($cars);
```

The following example sorts the elements of the $numbers array in ascending numerical order:

Example

```
$numbers = array(4, 6, 2, 22, 11);

sort($numbers);
```

Sort Array in Descending Order - rsort()

The following example sorts the elements of the $cars array in descending alphabetical order:

Example

```
$cars = array("Volvo", "BMW", "Toyota");

rsort($cars);
```

The following example sorts the elements of the $numbers array in descending numerical order:

Example

```php
$numbers = array(4, 6, 2, 22, 11);

rsort($numbers);
```

Sort Array (Ascending Order), According to Value - asort()

The following example sorts an associative array in ascending order, according to the value:

Example

```php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

asort($age);
```

Sort Array (Ascending Order), According to Key - ksort()

The following example sorts an associative array in ascending order, according to the key:

Example

```php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

ksort($age);
```

Sort Array (Descending Order), According to Value - arsort()

The following example sorts an associative array in descending order, according to the value:

Example

```php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
```

```php
arsort($age);
```

Sort Array (Descending Order), According to Key - krsort()

The following example sorts an associative array in descending order, according to the key:

Example

```php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

krsort($age);
```

PHP Multidimensional Arrays

In the previous pages, we have described arrays that are a single list of key/value pairs.

However, sometimes you want to store values with more than one key. For this, we have multidimensional arrays.

PHP - Multidimensional Arrays

A multidimensional array is an array containing one or more arrays.

PHP supports multidimensional arrays that are two, three, four, five, or more levels deep. However, arrays more than three levels deep are hard to manage for most people.

The dimension of an array indicates the number of indices you need to select an element.

- For a two-dimensional array you need two indices to select an element
- For a three-dimensional array you need three indices to select an element

---

PHP - Two-dimensional Arrays

A two-dimensional array is an array of arrays (a three-dimensional array is an array of arrays of arrays).

First, take a look at the following table:

Name	Stock	Sold
Volvo	22	18
BMW	15	13
Saab	5	2
Land Rover	17	15

We can store the data from the table above in a two-dimensional array, like this:

```
$cars = array (
  array("Volvo",22,18),
  array("BMW",15,13),
  array("Saab",5,2),
  array("Land Rover",17,15)
);
```

Now the two-dimensional $cars array contains four arrays, and it has two indices: row and column.

To get access to the elements of the $cars array we must point to the two indices (row and column):

Example

```
echo $cars[0][0].": In stock: ".$cars[0][1].", sold:
".$cars[0][2].".<br>";

echo $cars[1][0].": In stock: ".$cars[1][1].", sold:
".$cars[1][2].".<br>";

echo $cars[2][0].": In stock: ".$cars[2][1].", sold:
".$cars[2][2].".<br>";

echo $cars[3][0].": In stock: ".$cars[3][1].", sold:
".$cars[3][2].".<br>";
```

We can also put a `for` loop inside another `for` loop to get the elements of the $cars array (we still have to point to the two indices):

Example

```
for ($row = 0; $row < 4; $row++) {

  echo "<p><b>Row number $row</b></p>";

  echo "<ul>";

    for ($col = 0; $col < 3; $col++) {

      echo "<li>".$cars[$row][$col]."</li>";

    }

  echo "</ul>";

}
```

PHP Array Functions

PHP has a set of built-in functions that you can use on arrays.

Function	Description
array()	Creates an array
array_change_key_case()	Changes all keys in an array to lowercase or uppercase
array_chunk()	Splits an array into chunks of arrays
array_column()	Returns the values from a single column in the input array
array_combine()	Creates an array by using the elements from one "keys" array and one "values" array
array_count_values()	Counts all the values of an array
array_diff()	Compare arrays, and returns the differences (compare values only)
array_diff_assoc()	Compare arrays, and returns the differences (compare keys and values)
array_diff_key()	Compare arrays, and returns the differences (compare keys only)
array_diff_uassoc()	Compare arrays, and returns the differences (compare keys and values, using a user-defined comparison function)

array_diff_ukey()	Compare arrays, and returns the differences (compare keys only, using a user-defined key comparison function)
array_fill()	Fills an array with values
array_fill_keys()	Fills an array with values, specifying keys
array_filter()	Filters the values of an array using a callback function
array_flip()	Flips/Exchanges all keys with their associated values in an array
array_intersect()	Compare arrays, and returns the matches (compare values only)
array_intersect_assoc()	Compare arrays and returns the matches (compare keys and values)
array_intersect_key()	Compare arrays, and returns the matches (compare keys only)
array_intersect_uassoc()	Compare arrays, and returns the matches (compare keys and values, using a user-defined ke comparison function)
array_intersect_ukey()	Compare arrays, and returns the matches (compare keys only, using a user-defined key comparison function)
array_key_exists()	Checks if the specified key exists in the array

array_keys()	Returns all the keys of an array
array_map()	Sends each value of an array to a user-made function, which returns new values
array_merge()	Merges one or more arrays into one array
array_merge_recursive()	Merges one or more arrays into one array recursively
array_multisort()	Sorts multiple or multi-dimensional arrays
array_pad()	Inserts a specified number of items, with a specified value, to an array
array_pop()	Deletes the last element of an array
array_product()	Calculates the product of the values in an array
array_push()	Inserts one or more elements to the end of an array
array_rand()	Returns one or more random keys from an array
array_reduce()	Returns an array as a string, using a user-defined function

array_replace()	Replaces the values of the first array with the values from following arrays
array_replace_recursive()	Replaces the values of the first array with the values from following arrays recursively
array_reverse()	Returns an array in the reverse order
array_search()	Searches an array for a given value and returns the key
array_shift()	Removes the first element from an array, and returns the value of the removed element
array_slice()	Returns selected parts of an array
array_splice()	Removes and replaces specified elements of an array
array_sum()	Returns the sum of the values in an array
array_udiff()	Compare arrays, and returns the differences (compare values only, using a user-defined key comparison function)
array_udiff_assoc()	Compare arrays, and returns the differences (compare keys and values, using a built-in funct compare the keys and a user-defined function to compare the values)
array_udiff_uassoc()	Compare arrays, and returns the differences (compare keys and values, using two user-defin key comparison functions)

array_uintersect()	Compare arrays, and returns the matches (compare values only, using a user-defined key comparison function)
array_uintersect_assoc()	Compare arrays, and returns the matches (compare keys and values, using a built-in function compare the keys and a user-defined function to compare the values)
array_uintersect_uassoc()	Compare arrays, and returns the matches (compare keys and values, using two user-defined comparison functions)
array_unique()	Removes duplicate values from an array
array_unshift()	Adds one or more elements to the beginning of an array
array_values()	Returns all the values of an array
array_walk()	Applies a user function to every member of an array
array_walk_recursive()	Applies a user function recursively to every member of an array
arsort()	Sorts an associative array in descending order, according to the value
asort()	Sorts an associative array in ascending order, according to the value
compact()	Create array containing variables and their values

count()	Returns the number of elements in an array
current()	Returns the current element in an array
each()	Deprecated from PHP 7.2. Returns the current key and value pair from an array
end()	Sets the internal pointer of an array to its last element
extract()	Imports variables into the current symbol table from an array
in_array()	Checks if a specified value exists in an array
key()	Fetches a key from an array
krsort()	Sorts an associative array in descending order, according to the key
ksort()	Sorts an associative array in ascending order, according to the key
list()	Assigns variables as if they were an array
natcasesort()	Sorts an array using a case insensitive "natural order" algorithm

natsort()	Sorts an array using a "natural order" algorithm
next()	Advance the internal array pointer of an array
pos()	Alias of current()
prev()	Rewinds the internal array pointer
range()	Creates an array containing a range of elements
reset()	Sets the internal pointer of an array to its first element
rsort()	Sorts an indexed array in descending order
shuffle()	Shuffles an array
sizeof()	Alias of count()
sort()	Sorts an indexed array in ascending order
uasort()	Sorts an array by values using a user-defined comparison function and maintains the index association

uksort()	Sorts an array by keys using a user-defined comparison function
usort()	Sorts an array by values using a user-defined comparison function

PHP Form Handling

The PHP superglobals $_GET and $_POST are used to collect form-data.

PHP - A Simple HTML Form

The example below displays a simple HTML form with two input fields and a submit button:

Example

```
<html>
<body>

<form action="welcome.php" method="POST">
Name: <input type="text" name="name"><br>
E-mail: <input type="text" name="email"><br>
<input type="submit">
</form>

</body>
</html>
```

When the user fills out the form above and clicks the submit button, the form data is sent for processing to a PHP file named "welcome.php". The form data is sent with the HTTP POST method.

To display the submitted data you could simply echo all the variables.

The "welcome.php" looks like this:

```
<html>

<body>



Welcome <?php echo $_POST["name"]; ?><br>

Your email address is: <?php echo $_POST["email"]; ?>



</body>

</html>
```

The output could be something like this:

```
Welcome John

Your email address is john.doe@example.com
```

The same result could also be achieved using the HTTP GET method:

Example

Same example, but the method is set to GET instead of POST:

```
<html>

<body>



<form action="welcome_get.php" method="GET">

Name: <input type="text" name="name"><br>

E-mail: <input type="text" name="email"><br>
```

```
<input type="submit">

</form>



</body>

</html>
```

and "welcome_get.php" looks like this:

```
<html>

<body>



Welcome <?php echo $_GET["name"]; ?><br>

Your email address is: <?php echo $_GET["email"]; ?>



</body>

</html>
```

The code above is quite simple, and it does not include any validation.

You need to validate form data to protect your script from malicious code.

Think SECURITY when processing PHP forms!

This page does not contain any form validation, it just shows how you can send and retrieve form data.

However, the next pages will show how to process PHP forms with security in mind! Proper validation of form data is important to protect your form from hackers and spammers!

GET vs. POST

Both GET and POST create an array (e.g. array(key1 => value1, key2 => value2, key3 => value3, ...)). This array holds key/value pairs, where keys are the names of the form controls and values are the input data from the user.

Both GET and POST are treated as $_GET and $_POST. These are superglobals, which means that they are always accessible, regardless of scope - and you can access them from any function, class or file without having to do anything special.

$_GET is an array of variables passed to the current script via the URL parameters.

$_POST is an array of variables passed to the current script via the HTTP POST method.

When to use GET?

Information sent from a form with the GET method is **visible to everyone** (all variable names and values are displayed in the URL). GET also has limits on the amount of information to send. The limitation is about 2000 characters. However, because the variables are displayed in the URL, it is possible to bookmark the page. This can be useful in some cases.

GET may be used for sending non-sensitive data.

Note: GET should NEVER be used for sending passwords or other sensitive information!

When to use POST?

Information sent from a form with the POST method is **invisible to others** (all names/values are embedded within the body of the HTTP request) and has **no limits** on the amount of information to send.

Moreover POST supports advanced functionality such as support for multi-part binary input while uploading files to server.

However, because the variables are not displayed in the URL, it is not possible to bookmark the page.

Developers prefer POST for sending form data.

Next, lets see how we can process PHP forms the secure way!

PHP Form Validation

Think SECURITY when processing PHP forms!

These pages will show how to process PHP forms with security in mind. Proper validation of form data is important to protect your form from hackers and spammers!

The HTML form we will be working at in these chapters, contains various input fields: required and optional text fields, radio buttons, and a submit button:

The validation rules for the form above are as follows:

Field	Validation Rules
Name	Required. + Must only contain letters and whitespace
E-mail	Required. + Must contain a valid email address (with @ and .)
Website	Optional. If present, it must contain a valid URL
Comment	Optional. Multi-line input field (textarea)
Gender	Required. Must select one

First we will look at the plain HTML code for the form:

Text Fields

The name, email, and website fields are text input elements, and the comment field is a textarea.

The HTML code looks like this:

```
Name: <input type="text" name="name">

E-mail: <input type="text" name="email">

Website: <input type="text" name="website">

Comment: <textarea name="comment" rows="5" cols="40"></textarea>
```

Radio Buttons

The gender fields are radio buttons and the HTML code looks like this:

```
Gender:

<input type="radio" name="gender" value="female">Female

<input type="radio" name="gender" value="male">Male

<input type="radio" name="gender" value="other">Other
```

The Form Element

The HTML code of the form looks like this:

```
<form method="post" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

When the form is submitted, the form data is sent with method="post".

What is the `$_SERVER["PHP_SELF"]` variable?

The $_SERVER["PHP_SELF"] is a super global variable that returns the filename of the currently executing script.

So, the $_SERVER["PHP_SELF"] sends the submitted form data to the page itself, instead of jumping to a different page. This way, the user will get error messages on the same page as the form.

What is the htmlspecialchars() function?

The htmlspecialchars() function converts special characters into HTML entities. This means that it will replace HTML characters like < and > with < and >. This prevents attackers from exploiting the code by injecting HTML or Javascript code (Cross-site Scripting attacks) in forms.

Warning!

The $_SERVER["PHP_SELF"] variable can be used by hackers!

If PHP_SELF is used in your page then a user can enter a slash / and then some Cross Site Scripting (XSS) commands to execute.

Cross-site scripting (XSS) is a type of computer security vulnerability typically found in Web applications. XSS enables attackers to inject client-side script into Web pages viewed by other users.

Assume we have the following form in a page named "test_form.php":

```
<form method="post" action="<?php echo $_SERVER["PHP_SELF"];?>">
```

Now, if a user enters the normal URL in the address bar like "http://www.example.com/test_form.php", the above code will be translated to:

```
<form method="post" action="test_form.php">
```

So far, so good.

However, consider that a user enters the following URL in the address bar:

```
http://www.example.com/test_form.php/%22%3E%3Cscript%3Ealert('hacked')%3C/script%3E
```

In this case, the above code will be translated to:

```
<form method="post"
action="test_form.php/"><script>alert('hacked')</script>
```

This code adds a script tag and an alert command. And when the page loads, the JavaScript code will be executed (the user will see an alert box). This is just a simple and harmless example how the PHP_SELF variable can be exploited.

Be aware of that **any JavaScript code can be added inside the <script> tag!** A hacker can redirect the user to a file on another server, and that file can hold malicious code that can alter the global variables or submit the form to another address to save the user data, for example.

How To Avoid $_SERVER["PHP_SELF"] Exploits?

`$_SERVER["PHP_SELF"]` exploits can be avoided by using the `htmlspecialchars()` function.

The form code should look like this:

```
<form method="post" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

The `htmlspecialchars()` function converts special characters to HTML entities. Now if the user tries to exploit the PHP_SELF variable, it will result in the following output:

```
<form method="post"
action="test_form.php/&quot;&gt;&lt;script&gt;alert('hacked')&lt;/script&g
t;">
```

The exploit attempt fails, and no harm is done!

Validate Form Data With PHP

The first thing we will do is to pass all variables through PHP's `htmlspecialchars()` function.

When we use the `htmlspecialchars()` function; then if a user tries to submit the following in a text field:

```
<script>location.href('http://www.hacked.com')</script>
```

- this would not be executed, because it would be saved as HTML escaped code, like this:

```
&lt;script&gt;location.href('http://www.hacked.com')&lt;/script&gt;
```

The code is now safe to be displayed on a page or inside an e-mail.

We will also do two more things when the user submits the form:

1. Strip unnecessary characters (extra space, tab, newline) from the user input data (with the PHP `trim()` function)
2. Remove backslashes `\` from the user input data (with the PHP `stripslashes()` function)

The next step is to create a function that will do all the checking for us (which is much more convenient than writing the same code over and over again).

We will name the function `test_input()`.

Now, we can check each `$_POST` variable with the `test_input()` function, and the script looks like this:

Example

```php
// define variables and set to empty values
$name = $email = $gender = $comment = $website = "";


if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $name = test_input($_POST["name"]);
  $email = test_input($_POST["email"]);
  $website = test_input($_POST["website"]);
  $comment = test_input($_POST["comment"]);
  $gender = test_input($_POST["gender"]);
```

```
}


function test_input($data) {
  $data = trim($data);
  $data = stripslashes($data);
  $data = htmlspecialchars($data);
  return $data;
}
```

Run Example »

Notice that at the start of the script, we check whether the form has been submitted using `$_SERVER["REQUEST_METHOD"]`. If the `REQUEST_METHOD` is `POST`, then the form has been submitted - and it should be validated. If it has not been submitted, skip the validation and display a blank form.

However, in the example above, all input fields are optional. The script works fine even if the user does not enter any data.

The next step is to make input fields required and create error messages if needed.

PHP - Required Fields

From the validation rules table on the previous page, we see that the "Name", "E-mail", and "Gender" fields are required. These fields cannot be empty and must be filled out in the HTML form.

Field	Validation Rules
Name	Required. + Must only contain letters and whitespace

E-mail	Required. + Must contain a valid email address (with @ and .)
Website	Optional. If present, it must contain a valid URL
Comment	Optional. Multi-line input field (textarea)
Gender	Required. Must select one

In the previous chapter, all input fields were optional.

In the following code we have added some new variables: `$nameErr`, `$emailErr`, `$genderErr`, and `$websiteErr`. These error variables will hold error messages for the required fields. We have also added an `if else` statement for each `$_POST` variable. This checks if the `$_POST` variable is empty (with the PHP `empty()` function). If it is empty, an error message is stored in the different error variables, and if it is not empty, it sends the user input data through the `test_input()` function:

```
// define variables and set to empty values

$nameErr = $emailErr = $genderErr = $websiteErr = "";

$name = $email = $gender = $comment = $website = "";


if ($_SERVER["REQUEST_METHOD"] == "POST") {

  if (empty($_POST["name"])) {

    $nameErr = "Name is required";

  } else {

    $name = test_input($_POST["name"]);

  }
```

```php
  if (empty($_POST["email"])) {
    $emailErr = "Email is required";
  } else {
    $email = test_input($_POST["email"]);
  }

  if (empty($_POST["website"])) {
    $website = "";
  } else {
    $website = test_input($_POST["website"]);
  }

  if (empty($_POST["comment"])) {
    $comment = "";
  } else {
    $comment = test_input($_POST["comment"]);
  }

  if (empty($_POST["gender"])) {
    $genderErr = "Gender is required";
  } else {
    $gender = test_input($_POST["gender"]);
  }
}
```

PHP - Display The Error Messages

Then in the HTML form, we add a little script after each required field, which generates the correct error message if needed (that is if the user tries to submit the form without filling out the required fields):

Example

```
<form method="post" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>">



Name: <input type="text" name="name">

<span class="error">* <?php echo $nameErr;?></span>

<br><br>

E-mail:

<input type="text" name="email">

<span class="error">* <?php echo $emailErr;?></span>

<br><br>

Website:

<input type="text" name="website">

<span class="error"><?php echo $websiteErr;?></span>

<br><br>

Comment: <textarea name="comment" rows="5" cols="40"></textarea>

<br><br>

Gender:

<input type="radio" name="gender" value="female">Female

<input type="radio" name="gender" value="male">Male

<input type="radio" name="gender" value="other">Other
```

```
<span class="error">* <?php echo $genderErr;?></span>

<br><br>

<input type="submit" name="submit" value="Submit">



</form>
```

Run Example »

The next step is to validate the input data, that is "Does the Name field contain only letters and whitespace?", and "Does the E-mail field contain a valid e-mail address syntax?", and if filled out, "Does the Website field contain a valid URL?".

PHP Forms - Validate E-mail and URL

This chapter shows how to validate names, e-mails, and URLs.

PHP - Validate Name

The code below shows a simple way to check if the name field only contains letters, dashes, apostrophes and whitespaces. If the value of the name field is not valid, then store an error message:

```
$name = test_input($_POST["name"]);

if (!preg_match("/^[a-zA-Z-' ]*$/",$name)) {

  $nameErr = "Only letters and white space allowed";

}
```

The preg_match() function searches a string for pattern, returning true if the pattern exists, and false otherwise.

PHP - Validate E-mail

The easiest and safest way to check whether an email address is well-formed is to use PHP's `filter_var()` function.

In the code below, if the e-mail address is not well-formed, then store an error message:

```
$email = test_input($_POST["email"]);

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {

  $emailErr = "Invalid email format";

}
```

PHP - Validate URL

The code below shows a way to check if a URL address syntax is valid (this regular expression also allows dashes in the URL). If the URL address syntax is not valid, then store an error message:

```
$website = test_input($_POST["website"]);

if (!preg_match("/\b(?:(?:https?|ftp):\/\/|www\.)[-a-z0-
9+&@#\/%?=~_|!:,.;]*[-a-z0-9+&@#\/%=~_|]/i",$website)) {

  $websiteErr = "Invalid URL";

}
```

PHP - Validate Name, E-mail, and URL

Now, the script looks like this:

Example

```
// define variables and set to empty values
```

```php
$nameErr = $emailErr = $genderErr = $websiteErr = "";
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  if (empty($_POST["name"])) {
    $nameErr = "Name is required";
  } else {
    $name = test_input($_POST["name"]);
    // check if name only contains letters and whitespace
    if (!preg_match("/^[a-zA-Z-' ]*$/",$name)) {
      $nameErr = "Only letters and white space allowed";
    }
  }

  if (empty($_POST["email"])) {
    $emailErr = "Email is required";
  } else {
    $email = test_input($_POST["email"]);
    // check if e-mail address is well-formed
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
      $emailErr = "Invalid email format";
    }
  }

  if (empty($_POST["website"])) {
```

```
    $website = "";
  } else {
    $website = test_input($_POST["website"]);
    // check if URL address syntax is valid (this regular expression also
allows dashes in the URL)
    if (!preg_match("/\b(?:(?:https?|ftp):\/\/|www\.)[-a-z0-
9+&@#\/%?=~_|!:,.;]*[-a-z0-9+&@#\/%=~_|]/i",$website)) {
      $websiteErr = "Invalid URL";
    }
  }

  if (empty($_POST["comment"])) {
    $comment = "";
  } else {
    $comment = test_input($_POST["comment"]);
  }

  if (empty($_POST["gender"])) {
    $genderErr = "Gender is required";
  } else {
    $gender = test_input($_POST["gender"]);
  }
}
```

The next step is to show how to prevent the form from emptying all the input fields when the user submits the form.

This chapter shows how to keep the values in the input fields when the user hits the submit button.

PHP - Keep The Values in The Form

To show the values in the input fields after the user hits the submit button, we add a little PHP script inside the value attribute of the following input fields: name, email, and website. In the comment textarea field, we put the script between the `<textarea>` and `</textarea>` tags. The little script outputs the value of the `$name`, `$email`, `$website`, and `$comment` variables.

Then, we also need to show which radio button that was checked. For this, we must manipulate the checked attribute (not the value attribute for radio buttons):

```
Name: <input type="text" name="name" value="<?php echo $name;?>">


E-mail: <input type="text" name="email" value="<?php echo $email;?>">


Website: <input type="text" name="website" value="<?php echo $website;?>">


Comment: <textarea name="comment" rows="5" cols="40"><?php echo
$comment;?></textarea>


Gender:

<input type="radio" name="gender"

<?php if (isset($gender) && $gender=="female") echo "checked";?>

value="female">Female

<input type="radio" name="gender"

<?php if (isset($gender) && $gender=="male") echo "checked";?>

value="male">Male

<input type="radio" name="gender"
```

```
<?php if (isset($gender) && $gender=="other") echo "checked";?>

value="other">Other
```

PHP - Complete Form Example

Here is the complete code for the PHP Form Validation Example:

Example

Run Example »

PHP Form Validation Example

* required field

Name: *

E-mail: *

Website:

Comment:

Gender: Female Male Other *

Submit

Your Input: